

Worker crash on Supabase realtime reconnection (uncaught exception)

Incident report · TScopier worker · 2026-08-24

SEVERITY	High	STATUS	Open — fix proposed
DATES	Prod crashes Aug 23 23:04, Aug 24 09:17; staging down since Aug 24 13:10 UTC	AFFECTED	All user listeners during each crash window; staging listener still down
COMPONENT	worker/src/sessionManager.ts (realtime subscriptions)	ROOT CAUSE	Retry reuses an already-subscribed channel object; handler registration throws uncaught

1. Executive summary (plain English)

The worker keeps two background connections to Supabase's realtime service so it learns immediately when a user adds a channel or starts linking Telegram. When one of those connections drops — which happens on ordinary network interruptions or when Supabase restarts its infrastructure — the worker waits five seconds and reconnects. The recovery routine reconnects under the same connection name without discarding the broken connection object first. Supabase returns the old object, and adding message handlers to an already-connected object is not allowed, so the code throws an error nothing catches. Our worker treats any uncaught error as fatal and shuts down. Staging has been down since 13:10 UTC today because Railway did not restart it; production hit the same failure twice in the past day and only survived because its automatic restart policy fired. Every occurrence takes all user listeners offline until the process comes back. The fix is small and confined to the reconnection routine.

2. Issue encountered

- Staging listener (deployment `ac49e136` , commit `29cd0ee6`) entered `CRASHED` status at 2026-08-24T13:10:46Z and was not restarted by Railway.
- Fatal log lines:

```
[sessionManager] Realtime telegram_channels subscription CHANNEL_ERROR – retrying in 5s
[worker-fatal] UNCAUGHT_EXCEPTION Error: cannot add postgres_changes callbacks for realtime:telegram_channels_changes after subscribe().
The identical error followed for telegram_auth_pending_changes seconds later.
```

- Production (commit `54d82806`) logged the same `UNCAUGHT_EXCEPTION` at 2026-08-23T23:04:46Z and 2026-08-24T09:17:56Z. Railway's `ON_FAILURE` policy restarted it both times, but each crash disconnected every user listener until restart completed.
- This is not a new failure.** The retry logic was introduced on 2026-07-28 (`5ccf4276` , "Realtime subscription reconnect gap fix") and has been crashing the worker on every realtime drop since. The previous production deployment (`8b9f88f1`) recorded **17 identical crashes between Aug 20 16:05 and Aug 21 20:11**, each silently absorbed by automatic restarts. The defect went unnoticed because production always recovered within minutes and no alerting existed for `worker-fatal` messages.
- Both realtime subscriptions (`telegram_channels_changes` , `telegram_auth_pending_changes`) contain the same defect and can each trigger the crash independently.

3. Affected user(s)

ENVIRONMENT	DEPLOYMENT	WHEN	IMPACT
Production	7126d9eb (54d82806)	2026-08-23 23:04:46Z	All user listeners disconnected until automatic restart
Production	7126d9eb (54d82806)	2026-08-24 09:17:56Z	Same
Staging	ac49e136 (29cd0ee6)	2026-08-24 13:10:46Z	Listener fully down; not restarted — manual action required

No trades were lost — trade execution runs on a separate service. The impact is signal-listening downtime: signals sent during a crash window are delayed until the worker returns and catches up.

4. Root cause

In plain English: The worker holds two standing subscriptions to Supabase's realtime service. When one drops, the recovery code creates a replacement connection using the exact same name as the failed one, but never removes the failed one from the Supabase client first. Supabase identifies connections by name, so instead of a fresh connection it hands back the old, already-connected object. Registering message handlers on an object that is already connected is rejected by the library, and because this all happens inside a timer callback outside any error handling, the error escapes, reaches Node.js, and shuts the process down.

Technical detail: In `worker/src/sessionManager.ts` , both `subscribeToChannelChanges()` (:434) and `subscribeToAuthPendingChanges()` (:464):

- On `CHANNEL_ERROR` or `CLOSED` , the status callback sets the channel reference to `null` and schedules resubscription via `setTimeout(..., 5000)` (:458–459, :487–488).
- The errored channel is never passed to `supabase.removeChannel()` , so supabase-js keeps it registered under topic `realtime:telegram_channels_changes` .
- On retry, `supabase.channel('telegram_channels_changes')` deduplicates by topic and returns the existing subscribed instance. Calling `.on('postgres_changes', ...)` on it throws *"cannot add 'postgres_changes' callbacks after subscribe()* .
- The throw occurs inside a `setTimeout` callback with no try/catch anywhere up the stack → `uncaughtException` handler exits the process.

Trigger condition: any transient realtime disconnect (Supabase platform restart, network blip). Observed frequency: 17 crashes in 28 hours on the previous prod deployment; 2 more on the current one. The defect has existed since 2026-07-28.

Why it surfaced only now: it has been occurring continuously since July 28 — production simply recovered automatically each time, and nothing alerted on the crash. The Aug-24 staging event is only the first occurrence that was *not* auto-recovered, making the pattern visible.

Why restarts usually masked it: the defect lived only in the recovery path of a running process. A restart boots a fresh process with an empty Supabase client registry — no stale channel objects exist, so both subscriptions connect cleanly on first attempt and the worker resumes. Each process then ran fine until its next realtime drop, crashed, and was restarted again: production cycled healthy → drop → bad recovery → crash → clean restart roughly every 100 minutes during Aug 20–21. Restarts were an accidental safety net: users saw only brief listening gaps because `ON_FAILURE` relauches within seconds. Staging's Aug-24 crash became visible only because the automatic relaunch did not fire (reason not determinable from the Railway API; possibly retry-budget exhaustion or a platform-side issue). Two lasting lessons: uptime figures partly reflected fast crash-restart cycling rather than true stability, and the absence of alerting on `worker-fatal` let 17 crashes pass unnoticed — alerting is listed in follow-ups.

Why this diagnosis is certain: (1) The error text is emitted verbatim by supabase-js at exactly one place — its guard against registering `postgres_changes` handlers on a channel that has already been subscribed. (2) The log timing is deterministic: `CHANNEL_ERROR` at time T, fatal throw at T+5s, precisely the scheduled retry delay. (3) Across two deployments, all 36 recorded `CHANNEL_ERROR` events are matched 1:1 by an identical `UNCAUGHT_EXCEPTION` five seconds later; no crash of this class exists without a preceding drop. (4) The retry path is the only code location that re-registers handlers on a channel name that already errored.

5. The fix

Confined to `worker/src/sessionManager.ts` ; no behavior change beyond recovery:

1. **Remove before retry.** In each error branch, capture the channel reference and call `supabase.removeChannel(oldChannel)` *before* nulling the field and scheduling the retry. This guarantees the next attempt receives a fresh channel object.
2. **Guard the subscription body.** Wrap the body of both subscribe functions in try/catch, logging a warning and rescheduling on unexpected errors, so no future library change can escalate a recovery attempt into a process exit.
3. **Optional hardening.** Include a short random suffix in the channel name per attempt, making name reuse impossible regardless of client-side caching behavior.

Relation to the flood-wait work of 2026-08-21/24: none. That fix lives in `userListener.ts` polling logic; this defect predates it and is triggered by Supabase realtime drops, not Telegram throttling. Production crashed from this bug while already running the flood-wait fix, with no flood-wait activity involved at the time.

6. Files changed

Not yet implemented. Planned scope:

- `worker/src/sessionManager.ts` — remove-before-retry in both error branches (:456–459, :485–488); try/catch guards around both subscribe functions.
- No new unit tests: the failure requires a live socket state transition that cannot be reproduced in unit tests; verification is log-based (section 7).

7. Verification

- `npm --prefix worker run build` (typecheck) and full `npm --prefix worker test`.
- Deploy to staging; manually restart the crashed deployment.
- Confirm in Railway logs that subsequent realtime drops produce a clean removal-and-resubscribe sequence with no `UNCAUGHT_EXCEPTION`.
- On production, monitor for realtime `CHANNEL_ERROR` events over several days; absence of `worker-fatal` lines following them confirms the fix.

8. Deployment status

Investigation complete 2026-08-24. No code changed yet; nothing deployed. Immediate manual remediation required: redeploy/restart the staging listener. Given production has crashed twice in the past day, this fix should follow the normal path (dev → staging → prod) with minimal delay.

9. Follow-ups

1. Restart/redeploy the staging listener immediately (manual).
2. Implement the fix in `sessionManager.ts`, ship dev → staging, validate, promote to prod promptly.
3. Audit the codebase for other realtime retry loops that may reuse channel references.
4. Consider alerting on `worker-fatal UNCAUGHT_EXCEPTION` so crashes are noticed without waiting for user reports.
5. Unrelated but noted during the same review: Cerebras parser keys return HTTP 402 repeatedly — check quota/billing separately.